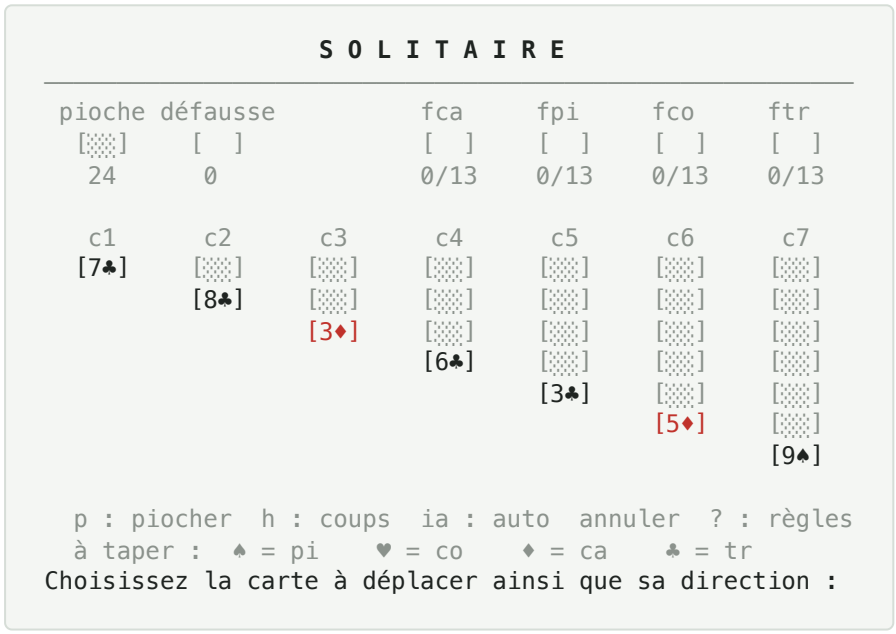
Le Solitaire

Dossier de soutenance

Mattias Mathevon et Jules Damour

Soutenance du vendredi 6 novembre 2026



Le plateau au démarrage de la partie de démonstration (choix 5 du menu) : 18 lignes sur 56 caractères.

Sommaire

1. Les structures de données : pioche, défausse, colonnes, fondations 2
2. Les commandes de jeu et les menus 5
3. Comment l'ordinateur propose les coups, et joue tout seul 7
4. Les points forts et les particularités du jeu 10
5. Les niveaux de jeu et le paramétrage 11
Annexes : les fichiers, les modules Python, lancer le jeu 12

1. Les structures de données

Le sujet interdit les classes. Tout le jeu tient donc dans les types vus en cours : des **textes** pour les cartes, des **listes** pour les piles, des **dictionnaires** pour ranger les piles et retrouver les cartes, et des **tuples** quand une fonction renvoie plusieurs valeurs.

1.1 Une carte : un petit texte

Chaque carte est un texte : sa **valeur**, suivie de sa **famille** en deux lettres.

Valeurs	`a` (as), `2` à `10`, `v` (valet), `d` (dame), `r` (roi)
Familles	`pi` pique ♠, `co` cœur ♥, `ca` carreau ♦, `tr` trèfle ♣
Exemples	`"7tr"` : 7 de trèfle, `"10co"` : 10 de cœur, `"dpi"` : dame de pique

Comme la famille fait toujours deux lettres, deux tranches suffisent pour séparer une carte : `carte[:-2]` donne la valeur (`"10co"` → `"10"`) et `carte[-2:]` donne la famille (`"10co"` → `"co"`). La couleur s'en déduit : une carte est rouge si sa famille est `"co"` ou `"ca"`.

Pour comparer deux valeurs, on cherche leur place dans la liste ordonnée `["a", "2", "3", …, "10", "v", "d", "r"]` avec `.index()` : le 7 est à la place 6 et le 8 à la place 7, donc le 7 est bien juste en dessous du 8.

Le **paquet** est la liste des 52 cartes (`jeu_depart.py`). Pour une nouvelle partie, on le copie avec `list()` puis on mélange la copie avec `random.shuffle()`. Il sert aussi à vérifier qu'une carte tapée par le joueur existe.

1.2 Les 13 piles dans un seul dictionnaire : `piles`

Toutes les piles du jeu sont rangées dans un dictionnaire : la clé est le nom de la pile, la valeur est la liste de ses cartes. Voici la donne de la partie de démonstration :

```
piles = {
    #la pioche : 24 cartes, la prochaine carte piochée est la dernière
    "p": ["atr", "2tr", "7pi", "vca", ..., "aca", "5tr", "7co"],
    #la défausse, vide au début
    "d": [],
    #les colonnes : de la première carte posée jusqu'à la carte visible
    "c1": ["7tr"],
    "c2": ["dco", "8tr"],
    ...
    "c7": ["rpi", "api", "8ca", "9co", "3pi", "dtr", "9pi"],
    #les fondations : une par famille, vides au début
    "fca": [], "fpi": [], "fco": [], "ftr": [],
}
```

Une règle vaut pour toutes les piles : **la dernière carte de la liste est la carte du dessus**. On la lit avec `piles[nom][-1]`, on la retire avec `pop()` et on pose une carte avec `append()`.

Élément	Clé	Ce que contient la liste	Au départ
Pioche	`"p"`	les cartes face cachée ; la dernière est la prochaine piochée	24 cartes
Défausse	`"d"`	les cartes piochées ; seule la dernière est jouable	vide
Colonnes	`"c1"` à `"c7"`	de la première carte posée (indice 0) jusqu'à la carte du dessus, la seule qu'on voit en entier	1 à 7 cartes
Fondations	`"fca"`, `"fpi"`, `"fco"`, `"ftr"`	une famille par fondation, de l'as (indice 0) jusqu'au roi	vides

La pioche et la défausse. Taper `p` fait un `pop()` sur la pioche et un `append()` sur la défausse, une à trois fois selon le niveau. Quand la pioche est vide, `p` remet toute la défausse dans la pioche en reprenant les cartes par le dessus : les cartes retrouvent l'ordre qu'elles avaient avant d'être piochées. À l'écran, la défausse montre ses 1, 2 ou 3 dernières cartes selon le niveau, avec `piles["d"][-difficulte:]` (partie 5).

Les colonnes. À l'écran, une colonne s'affiche de haut en bas dans l'ordre de la liste : la carte d'indice 0 est en haut, la dernière carte, celle qu'on peut jouer, est en bas. Pour dessiner les 7 colonnes côte à côte, on écrit l'écran ligne par ligne : la ligne `n` affiche la carte d'indice `n` de chaque colonne.

Les fondations. Le nom d'une fondation se termine par sa famille : `destination[-2:]` donne directement la famille attendue (`"fca"` → `"ca"`). Une fondation est complète à 13 cartes, et la partie est gagnée quand les quatre le sont (`verif_gagnee.py`).

1.3 Les cartes cachées : `cachees`

Dans une colonne, les cartes face cachée sont toujours en dessous, et les cartes visibles au dessus. Un seul nombre par colonne suffit donc : combien de cartes sont cachées.

```
cachees = {"c1": 0, "c2": 1, "c3": 2, "c4": 3, "c5": 4, "c6": 5, "c7": 6}
```

Par exemple, `cachees["c4"]` vaut 3 : les cartes d'indice 0, 1 et 2 de c4 sont cachées, seule celle d'indice 3 est visible. Le même test sert à l'affichage et aux vérifications : **une carte est cachée si sa position est plus petite que `cachees[colonne]`**. Quand un déplacement ne laisse plus que des cartes cachées dans une colonne, on retire 1 à ce nombre : la carte du dessus se retourne.

1.4 Où est chaque carte : `index`

`index` associe chaque carte au nom de la pile où elle se trouve :

```
index = {"7tr": "c1", "dco": "c2", "8tr": "c2", ..., "5ca": "c6", ..., "7co": "p"}
```

Quand le joueur tape `5ca-c4`, `index["5ca"]` donne tout de suite la pile de départ, `"c6"`, sans chercher la carte dans les 13 piles. Ensuite `piles["c6"].index("5ca")` donne sa position dans la colonne. En contrepartie, il faut tenir `index` à jour : à chaque déplacement, chaque carte déplacée reçoit sa nouvelle pile, et chaque carte piochée passe à `"d"`.

1.5 Un coup pas à pas : `5ca-c4`

Au départ de la partie de démonstration, le joueur tape `5ca-c4` pour poser le 5♦ (la carte visible de c6) sur le 6♣ (la carte visible de c4).

1. `verif_coup()` découpe la saisie avec `split("-")` et renvoie le tuple `("5ca", "c4")`.
2. `index["5ca"]` donne le départ, `"c6"`, et `piles["c6"].index("5ca")` donne la position : 5.

3. Le morceau à déplacer est `piles["c6"][5:]`, c'est-à-dire la carte et toutes celles posées dessus : ici `["5ca"]`.
4. Les vérifications passent : la carte est visible (5 n'est pas plus petit que `cachees["c6"]`, qui vaut 5), le 5♦ rouge va sur un 6♣ noir, et 5 est juste en dessous de 6.
5. `del piles["c6"][5:]` retire le morceau, `piles["c4"].extend(morceau)` le pose sur c4, et `index["5ca"]` devient `"c4"`.
6. c6 n'a plus que 5 cartes, toutes cachées : `cachees["c6"]` passe de 5 à 4, et le 3♥ se retourne.

	Avant	Après
`piles["c4"]`	`["dpi", "6pi", "4ca", "6tr"]`	`["dpi", "6pi", "4ca", "6tr", "5ca"]`
`piles["c6"]`	`["vpi", "6co", "2pi", "rca", "3co", "5ca"]`	`["vpi", "6co", "2pi", "rca", "3co"]`
`cachees["c6"]`	`5`	`4`
`index["5ca"]`	`"c6"`	`"c4"`

```
S O L I T A I R E
──────────────────────────────────────────────────
pioche défausse          fca     fpi     fco     ftr
[▒▒]    [  ]            [  ]    [  ]    [  ]    [  ]
 24      0              0/13    0/13    0/13    0/13

 c1      c2      c3      c4      c5      c6      c7
[7♣]    [▒▒]    [▒▒]    [▒▒]    [▒▒]    [▒▒]    [▒▒]
        [8♣]    [▒▒]    [▒▒]    [▒▒]    [▒▒]    [▒▒]
                [3♦]    [▒▒]    [▒▒]    [▒▒]    [▒▒]
                        [6♣]    [▒▒]    [▒▒]    [▒▒]
                        [5♦]    [3♣]    [3♥]    [▒▒]
                                                [▒▒]
                                                [9♠]

p : piocher  h : coups  ia : auto  annuler  ? : règles
à taper :  ♠ = pi    ♥ = co    ♦ = ca    ♣ = tr
```

Le plateau juste après `5ca-c4` : le 5♦ est posé sur le 6♣ en c4, et en c6 le 3♥ s'est retourné.

À comparer avec le plateau de départ, sur la page de garde.

1.6 Les autres données

- **Le nombre de coups** (`nb_coups`) et **le niveau** (`difficulte` : 1, 2 ou 3) sont de simples entiers, gardés par `main.py`. Pour le jeu automatique, `main.py` garde aussi `auto` (`True` quand l'ordinateur joue tout seul) et `nb_tours` (partie 3).
- **La sauvegarde** est un dictionnaire `{"piles": …, "cachees": …, "index": …, "nbr_coup": …, "difficulte": …}` écrit dans `sauvegarde.json` avec le module `json`.
- **La copie pour annuler.** Avant chaque coup, `main.py` copie la partie. Chaque liste de `piles` est copiée une par une avec `list()`, sinon la copie et la partie partageraient les mêmes listes de cartes. Pour `cachees` et `index`, `dict()` suffit : leurs valeurs sont des nombres et des textes, qu'on ne modifie jamais sur place.
- **Les coups possibles** sont une liste de textes, comme `["aca-fca", "5ca-c4", "p"]`, rangée du plus utile au moins utile (partie 3).

2. Les commandes de jeu et les menus

2.1 Le menu de départ

```
                S O L I T A I R E
-------------------------------------------------

   1 : nouvelle partie
   2 : reprendre la partie sauvegardée
   3 : difficulté (actuellement : facile)
   4 : quitter
   5 : partie de démonstration (toujours la même donne)

Votre choix : 5
```

Choix	Ce qui se passe
1	Nouvelle partie : le paquet est mélangé puis distribué (1 carte en c1, 2 en c2, … 7 en c7, les 24 autres dans la pioche).
2	Reprend la partie sauvegardée, avec son nombre de coups et son niveau. Sans sauvegarde : « Aucune partie sauvegardée », puis retour au menu.
3	Choix du niveau (partie 5), puis retour au menu.
4	Quitter.
5	Partie de démonstration : toujours la même donne.

Le choix est vérifié : tant que le joueur ne tape pas un nombre de 1 à 5, le menu redemande. Le menu affiche aussi le niveau en cours.

2.2 Les commandes pendant la partie

Chaque commande se tape en une seule saisie, suivie d'Entrée. Le rappel des commandes et des codes des familles reste affiché sous le plateau.

Saisie	Effet	Compte comme un coup
`carte-pile` ex : `5ca-c4`	déplace la carte, et toutes celles posées dessus, vers la pile	oui
`p`	pioche 1, 2 ou 3 cartes selon le niveau ; si la pioche est vide, remet la défausse dans la pioche	non
`h`	affiche les coups possibles, du plus utile au moins utile (partie 3)	non
`ia`	l'ordinateur joue tout seul, jusqu'à gagner ou jusqu'à ne plus trouver de coup utile (partie 3)	oui, chacun de ses coups
`annuler`	remet la partie comme avant le dernier coup, ou la dernière pioche	le compteur revient à sa valeur d'avant
`sauvegarde`	enregistre la partie pour la reprendre plus tard (choix 2 du menu)	non
`?`	affiche les règles du jeu et toutes les commandes	non

2.3 Écrire un déplacement

Un déplacement s'écrit **carte-pile** : la carte, un tiret, puis la pile d'arrivée (`c1` à `c7` , ou une fondation : `fca` , `fpi` , `fco` , `ftr`). Le joueur n'a pas besoin de dire d'où part la carte : `index` le sait. Le même format sert pour tous les déplacements : depuis une colonne, la défausse ou une fondation, vers une colonne ou une fondation.

Plusieurs cartes d'un coup. On tape la première carte du groupe à déplacer, la plus haute à l'écran : toutes les cartes posées dessus partent avec elle, car le morceau déplacé est `piles[depart][position:]` . On peut ainsi

déplacer toutes les cartes visibles d'une colonne, ou seulement une partie. Par exemple, si une colonne montre 9♠, 8♥, 7♣, taper `8co-c2` emmène le 8♥ et le 7♣, et laisse le 9♠ en place (si c2 se termine par le 9♣).

2.4 Toutes les saisies sont vérifiées

Un déplacement passe par quatre vérifications, dans cet ordre. Dès que l'une échoue, le jeu affiche la raison, rien ne bouge, et le coup n'est pas compté. Les exemples sont pris au départ de la partie de démonstration.

Vérification	Ce qui est contrôlé	Exemples refusés
1. La saisie `verif_coup.py`	un seul tiret, une carte qui existe, une pile d'arrivée qui existe	`5ca c4` (pas de tiret) `xyz-c4`, `5ca-c9`
2. La carte `verif_carte.py`	elle n'est ni dans la pioche, ni face cachée ; dans la défausse ou une fondation, c'est celle du dessus ; une seule carte à la fois vers une fondation	`7co-c2` (encore dans la pioche) `dpi-c1` (face cachée)
3. Vers une colonne `verif_colonne.py`	sur une colonne vide, seulement un roi ; sinon une couleur différente et une valeur juste en dessous	`6tr-c2` (noir sur noir) `3ca-c2` (3♦ sur 8♣)
4. Vers une fondation `verif_fondations.py`	la bonne famille ; un as sur une fondation vide ; sinon la valeur juste au dessus	`3tr-fpi` (mauvaise famille) `3ca-fca` (ce n'est pas un as)

Le menu et le choix du niveau vérifient aussi leurs saisies : ils redemandent tant que la réponse n'est pas dans la liste des choix possibles.

3. Comment l'ordinateur propose les coups, et joue tout seul

Quand le joueur tape `h`, l'ordinateur affiche tous les coups jouables, **classés du plus utile au moins utile**. Avec `ia`, il joue tout seul en suivant ce classement. Le code est dans `coup_iav1.py` (`trouver_coups()` et `afficher_coups()`) et dans la boucle de `main.py`.

3.1 Trouver les coups : essayer chaque carte sur chaque pile

L'ordinateur ne devine rien : il essaie toutes les combinaisons et garde celles que le jeu accepterait.

1. Il parcourt les 52 cartes de `index`. Une carte encore dans la pioche est passée, puisqu'elle est face cachée. Pour les autres, il calcule la pile de départ, la position et le morceau, exactement comme pour un vrai coup.
2. Pour chaque carte, il essaie les 11 piles d'arrivée (c1 à c7 et les 4 fondations), sauf celle où la carte est déjà.
3. Il appelle `verif_carte()`, puis `verifier_fondation()` si l'arrivée est une fondation, ou `verif_colonne()` si c'est une colonne. Si tout est bon, le coup `"carte-pile"` est gardé.
4. S'il reste des cartes dans la pioche ou dans la défausse, il ajoute `p`.

Cela fait au plus $52 \times 11 = 572$ essais : la réponse est immédiate. Si aucun coup n'existe, le jeu affiche « Aucun coup possible. ».

3.2 Les mêmes vérifications que pour le joueur

Plutôt que d'écrire les règles une deuxième fois, l'ordinateur réutilise les trois fonctions de vérification du joueur. On leur a ajouté un paramètre `parler` : avec `parler=False`, elles répondent `True` ou `False` sans afficher leur message de refus. Sans ça, `h` afficherait des dizaines de messages, un par essai raté. Pendant le jeu, `parler` vaut `True` par défaut, donc les messages s'affichent normalement.

> L'avantage : **un coup proposé par l'ordinateur est forcément accepté par le jeu**, puisque c'est le même code qui décide. Et si on corrige une règle, l'ordinateur suit tout seul.

3.3 Classer les coups, du plus utile au moins utile

Chaque coup trouvé est rangé dans une liste selon son type. À la fin, les listes sont mises bout à bout avec `+`, dans cet ordre :

Rang	Type de coup	Pourquoi	Liste
1	une carte monte sur sa fondation	c'est le but du jeu	`vers_fondation`
2	la carte part et une carte cachée se retourne	une carte de plus à jouer	`retourne_carte`
3	une carte de la défausse va sur une colonne	la pioche se vide	`depuis_defausse`
4	une carte change de colonne et libère celle d'en dessous pour sa fondation	prépare un rang 1	`libere_carte`
5	toute une colonne part : un roi attend une place, et aucune colonne n'est vide	fait de la place au roi	`vide_colonne`
6	piocher : `p`	une nouvelle carte arrive	
7	les autres changements de colonne	la carte change de place	`entre_colonnes`
8	une carte redescend d'une fondation	la partie recule	`depuis_fondation`

Pour reconnaître le rang 2, on regarde si la carte qui part est la première carte visible d'une colonne qui a des cartes cachées : `position == cachees[depart]` et `position > 0`. Pour le rang 4, on teste la carte d'en dessous

avec `verifier_fondation()`. Pour le rang 5, un roi « attend » quand il est posé sur des cartes cachées, ou sur le dessus de la défausse.

Deux choix volontaires : **la pioche reste secrète** (`h` propose `p` sans dire quelle carte va sortir), et **un roi qui est déjà la première carte de sa colonne** n'est pas proposé vers une autre colonne vide, ça ne changerait rien. Il peut toujours monter sur sa fondation.

```
                    S O L I T A I R E
 ──────────────────────────────────────────────────────────

 pioche défausse          fca     fpi     fco     ftr
  [ ]    [7♠]             [6♦]    [ ]     [ ]     [5♣]
   0      7               6/13    0/13    0/13    5/13

   c1      c2      c3      c4      c5      c6      c7
  [r♦]    [d♥]    [d♦]    [d♠]    [▒▒]    [▒▒]    [▒▒]
  [d♣]            [v♣]    [v♦]    [▒▒]    [6♥]    [▒▒]
  [v♥]            [10♦]           [8♠]    [5♠]    [▒▒]
                  [9♠]            [7♦]    [4♥]    [9♥]
                  [8♥]            [6♠]    [3♠]    [8♣]
                  [7♣]            [5♥]    [2♥]    [7♥]
                                  [4♠]            [6♣]
                                  [3♥]
                                  [2♠]

  p : piocher  h : coups  ia : auto  annuler  ? : règles
  à taper :  ♠ = pi    ♥ = co    ♦ = ca    ♣ = tr
Choisissez la carte à déplacer ainsi que sa direction : h

Coups possibles, du plus utile au moins utile :
    6tr-ftr
    6co-c3
    p
    5co-c7
    vtr-c2
    6ca-c3
Appuyez sur Entrée pour continuer
```

Exemple, au milieu de notre partie gagnante (après 41 coups et 24 pioches) :

- `6tr-ftr` monte le 6♣ sur sa fondation (rang 1) ;
- `6co-c3` pose le 6♥ sur le 7♣ et retourne la carte cachée de c6 (rang 2) ;
- puis `p` (rang 6) ;
- `5co-c7` et `vtr-c2` changent seulement des cartes de colonne (rang 7) ;
- `6ca-c3` ferait redescendre le 6♦ de sa fondation (rang 8).

Après Entrée, le plateau se réaffiche. Afficher les coups ne compte pas comme un coup.

3.4 L'ordinateur joue tout seul : la commande `ia`

Quand le joueur tape `ia`, `main.py` met `auto` à `True`. À chaque tour de la boucle de jeu, au lieu de demander un coup au joueur, il appelle `trouver_coups(piles, cachees, index, seulement_utiles=True)` et joue **le premier coup de la liste**. Il le joue avec la même fonction `deplacement()` que le joueur : le coup lui est donné dans le paramètre `coup_ordi`, au lieu d'être lu avec `input()`. Chaque coup reste affiché 0,2 seconde, avec « L'ordinateur a joué : … ». Les coups comptent et `annuler` marche comme d'habitude.

- **Pas d'allers-retours.** Avec `seulement_utiles=True`, la liste s'arrête après `p` : l'ordinateur ne joue jamais les rangs 7 et 8. Sinon il pourrait déplacer un 7 noir d'un 8 rouge à l'autre, puis revenir, sans fin. Chaque coup qu'il joue fait donc avancer la partie : une carte monte, une carte se retourne, la pioche diminue, ou l'un de ces coups se prépare.
- **Pas de pioche à l'infini.** `nb_tours` compte combien de fois il a retourné la défausse sans rien jouer d'autre. S'il doit la retourner une deuxième fois, c'est qu'il a vu toutes les cartes de la pioche sans pouvoir en placer une : il s'arrête, affiche « L'ordinateur ne trouve plus de coup utile, à vous de jouer » et rend la main.

```
                    S O L I T A I R E
 pioche défausse          fca     fpi     fco     ftr
  [▒▒]    [5♠]            [3♦]    [  ]    [  ]    [  ]
   8       5              3/13    0/13    0/13    0/13

   c1      c2      c3      c4      c5      c6      c7
  [7♣]    [▒▒]    [d♦]    [d♠]    [▒▒]    [▒▒]    [▒▒]
  [6♦]    [8♣]    [v♣]            [▒▒]    [▒▒]    [▒▒]
  [5♣]    [7♥]    [10♦]           [8♠]    [▒▒]    [▒▒]
  [4♦]    [6♣]    [9♠]            [7♦]    [r♦]    [▒▒]
  [3♣]    [5♦]    [8♥]            [6♠]    [d♣]    [3♠]
  [2♥]    [4♣]                    [5♥]
          [3♥]                    [4♠]

  p : piocher  h : coups  ia : auto  annuler  ? : règles
  à taper :  ♠ = pi    ♥ = co    ♦ = ca    ♣ = tr
L'ordinateur a joué : 8co-c3
```

La partie de démonstration après `ia` : l'ordinateur vient de poser le 8♥ sur le 9♠ en c3.

Résultats. L'ordinateur gagne la partie de démonstration aux trois niveaux (en 91 coups au niveau facile). Sur 500 parties mélangées au hasard, il en gagne 41 % au niveau facile, 21 % au niveau moyen et 11 % au niveau difficile.

3.5 La fin automatique, comme dans le vrai solitaire

Dans le vrai solitaire, la partie est gagnée dès que toutes les cartes sont retournées : le jeu range alors tout seul les cartes dans les fondations. Notre jeu fait pareil. Au début de chaque tour, `tout_est_retourne()` (`verif_gagnee.py`) regarde s'il n'y a plus aucune carte cachée dans les colonnes, et si la pioche et la défausse sont vides. Si oui, `main.py` affiche « Toutes les cartes sont retournées, la partie est gagnée » et passe `auto` à `True` : l'ordinateur monte les cartes une à une, jusqu'à la victoire.

Pourquoi c'est forcément gagné. Prenons la plus petite carte qui n'est pas encore en fondation. Elle est forcément sur le dessus de sa colonne, car les cartes posées sur elle seraient encore plus petites. Et toutes les cartes plus petites de sa famille sont déjà sur la fondation : elle peut donc monter. On recommence jusqu'à la dernière carte. On attend que la pioche soit vide, sinon au niveau difficile une carte coincée dans la pioche pourrait ne jamais sortir.

4. Les points forts et les particularités du jeu

- **L'ordinateur sait jouer tout seul.** Avec `ia`, il suit le même classement que `h` en évitant les coups inutiles, et il gagne la partie de démonstration aux trois niveaux. Quand il est bloqué, il rend la main au joueur au lieu de tourner en rond.
- **La fin automatique**, comme dans le vrai solitaire : dès que tout est retourné et que la pioche est finie, les cartes montent toutes seules dans les fondations.
- **Un coup, une saisie.** Tous les déplacements s'écrivent `carte-pile`, y compris pour déplacer plusieurs cartes d'un coup. Le joueur n'a jamais à dire d'où part la carte.
- **Chaque refus est expliqué.** Le jeu dit pourquoi un coup est impossible (carte cachée, même couleur, mauvaise famille…), sans rien changer à la partie.
- **Annuler le dernier coup.** `annuler` remet la partie comme avant le dernier coup, même si c'était une pioche ou un coup de l'ordinateur, et le compteur de coups revient en arrière. Pour cela, `main.py` copie la partie avant chaque coup. On ne peut annuler qu'un coup : un deuxième `annuler` affiche « Il n'y a rien à annuler ».
- **Sauvegarder et reprendre.** `sauvegarde` enregistre toute la partie (piles, cartes cachées, index, nombre de coups et niveau) dans un fichier JSON. Le choix 2 du menu la reprend exactement où elle en était.
- **Un plateau compact et lisible.** Au démarrage, le jeu tient en 18 lignes sur 56 caractères, sous la limite de 20 × 60. Les cœurs et les carreaux sont en rouge, les cartes cachées s'affichent `[▒▒]` et les emplacements vides `[  ]`. On voit le nombre de cartes de la pioche et de la défausse, et le remplissage de chaque fondation (sur 13). La défausse s'affiche en éventail, comme dans le vrai solitaire (partie 5). L'écran est effacé avant chaque affichage : le plateau reste toujours au même endroit.
- **De l'aide à tout moment.** Le rappel des commandes sous le plateau, l'écran des règles avec `?`, et les coups possibles classés avec `h`.
- **Une partie de démonstration et une partie gagnante.** Le choix 5 donne toujours la même donne. Le fichier `partie_gagnante.txt` contient les saisies qui la gagnent en 95 coups : `python3 main.py < partie_gagnante.txt` rejoue toute la partie jusqu'à la victoire. Après chaque modification, on s'en est servi pour vérifier qu'on n'avait rien cassé.
- **Une animation de victoire.** « VICTOIRE » s'affiche en grandes lettres qui changent de couleur, suivi du nombre de coups joués.
- **Uniquement des listes, des dictionnaires et des tuples**, comme le demande le sujet : aucune classe.
- **Un fichier par grande fonction, et git pour travailler à deux.** Chaque partie du jeu a son fichier (affichage, pioche, vérifications, sauvegarde…, voir l'annexe A). Chacun travaillait sur sa branche GitHub, puis on fusionnait par pull request : on a pu avancer en même temps sans écraser le travail de l'autre.

5. Les niveaux de jeu et le paramétrage

Le niveau se choisit avec le choix 3 du menu, avant de lancer la partie. Il fixe le **nombre de cartes tirées à chaque pioche**.

```
   1 : facile    (1 carte à chaque pioche)
   2 : moyen     (2 cartes à chaque pioche)
   3 : difficile (3 cartes à chaque pioche)
Votre niveau : 3
```

Niveau	Ce que ça change
1 : facile par défaut	les cartes de la pioche passent une par une sur la défausse, et chacune peut être jouée
2 : moyen	2 cartes à chaque pioche : seule celle du dessus est jouable, l'autre reste bloquée tant que la première n'est pas jouée
3 : difficile	3 cartes à chaque pioche : deux sur trois sont bloquées

La défausse en éventail. Comme dans le vrai solitaire, l'écran montre les cartes de la dernière pioche : 1 au niveau facile, 2 au niveau moyen, 3 au niveau difficile. Seule la dernière, à droite et en couleur, peut être jouée ; les autres sont en gris. `afficher_defausse()` prend `piles["d"][-difficulte:]`, et compte la largeur des cartes pour que les fondations restent à leur place.

```
pioche défausse          fca     fpi     fco     ftr
 [▒▒]    [9♦][v♣][2♦]    [  ]    [  ]    [  ]    [  ]
  18      6              0/13    0/13    0/13    0/13
```

Niveau difficile, après deux pioches : le 9♦ et le valet de trèfle sont visibles mais bloqués, seul le 2♦ est jouable.

Pourquoi c'est plus dur. Avec 24 cartes et 3 cartes par pioche, si le joueur ne joue aucune carte de la défausse, ce sont toujours les 8 mêmes cartes qui arrivent sur le dessus, tour de pioche après tour de pioche. Pour atteindre les autres, il faut jouer des cartes de la défausse : ça décale toute la suite de la pioche. On le voit avec l'ordinateur : sur 500 parties, il en gagne 41 % au niveau facile, mais 11 % seulement au niveau difficile.

Dans le code, le niveau est un simple entier, `difficulte`, égal au nombre de cartes à tirer. `choisir_difficulte()` renvoie `int(choix)`, et `piocher()` s'en sert directement dans sa boucle `while i <= difficulte`. La boucle s'arrête plus tôt si la pioche se vide en cours de route.

Le niveau est gardé dans la sauvegarde : une partie reprise continue au même niveau. Le menu affiche toujours le niveau en cours (« difficulté (actuellement : facile) »).

Autres réglages. La pioche peut être retournée autant de fois qu'on veut. La partie de démonstration se joue au niveau choisi ; notre partie gagnante en 95 coups est prévue pour le niveau facile.

Annexes

A. Les fichiers du programme

Fichier	Rôle
`main.py`	lance le jeu : menu, boucle de la partie, compteur de coups, annuler, et le jeu automatique (`ia` et fin automatique)
`menu.py`	le menu de départ et le choix du niveau
`jeu_depart.py`	le paquet de 52 cartes, la donne de démonstration, et `nouvelle_partie()` qui mélange et distribue
`affichage.py`	le plateau (avec la défausse en éventail), les couleurs et l'écran des règles
`deplacement.py`	lit la commande du joueur (ou reçoit celle de l'ordinateur), lance les vérifications et déplace les cartes
`verif_coup.py`	vérifie la forme de la saisie `carte-pile`
`verif_carte.py`	vérifie que la carte a le droit de bouger
`verif_colonne.py`	les règles pour poser sur une colonne
`verif_fondations.py`	les règles pour poser sur une fondation
`pioche.py`	la commande `p`
`coup_iav1.py`	trouve les coups possibles et les classe par priorité : sert à `h` et à l'ordinateur quand il joue
`sauvegarder.py`	la sauvegarde et la reprise de la partie
`verif_gagnee.py`	détecte la victoire (4 fondations de 13 cartes), et le moment où tout est retourné
`victoire.py`	l'animation de victoire
`partie_gagnante.txt`	les saisies qui gagnent la partie de démonstration

B. Les modules Python utilisés

Module	À quoi il sert
`random`	`shuffle()` : mélanger le paquet pour une nouvelle partie
`json`	`dump()` et `load()` : écrire et relire la sauvegarde
`os`	`system("")` : activer les couleurs dans le terminal de Windows ; `path.exists()` : savoir si une sauvegarde existe
`time`	`sleep()` : laisser le temps de voir chaque coup de l'ordinateur, et faire tourner les couleurs de l'animation de victoire

Ces quatre modules sont fournis avec Python : il n'y a rien à installer.

C. Lancer le jeu

Dans un terminal : `python3 main.py` (sous Windows : `python main.py`). Pour rejouer la partie gagnante d'un coup : `python3 main.py < partie_gagnante.txt`.